

Tutorial 5

Introduction To Programming

Variable Length Arguments

Default Parameter

Suppose a function has been defined that returns value at a point x, given the slope and intercept of a line.

Example

```
def value(x, slope, intercept):  
    return (x*slope + intercept)  
  
x = 3  
m = 2  
c = 1  
  
print(value(6, m, c))
```

Output

13

But what if we forget to pass the intercept value? It definitely should show an error!!

```
## Default parameter:  
  
def value(x, slope, intercept):  
    return (x*slope + intercept)  
  
x = 3  
m = 2  
c = 1  
  
print(value(5, m))  
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-4-af5fbff1b763> in <module>()  
      8 c = 1  
      9  
>>> 10 print(value(5, m))  
  
TypeError: value() missing 1 required positional argument: 'intercept'
```

SEARCH STACK OVERFLOW

Sometimes, To handle such cases, where we want some value to be pre-assigned until and unless we want so, we use what we call 'default parameter'.
example:

```
## Default parameter:

def value(x, slope, intercept = 1):
    return (x*slope + intercept)

x = 3
m = 2

print('output:', value(5, m))

output: 11
```

Here, the intercept has not been provided in the argument during the function call, as it has already been assigned as 1.

But what if we want to change the value of the intercept? Can we pass a new value to it?

```
## Default parameter:

def value(x, slope, intercept = 1):
    return (x*slope + intercept)

x = 3
m = 2

print('output:', value(5, m, 4))

output: 14
```

Yes!! It still does work fine. That's one of the use case of default parameters.

Is a way to pass a variable number of arguments(or parameters to the function)?

introduction to **"*args"**. It is used to pass a variable number of arguments to the function.

“***args**” is basically used to pass arguments as tuple and the fun fact is, these passed arguments also form a tuple inside the function. This tuple has the name “**args**”[EXCLUDING the asteriks].

For example, if you have to use a function that sums up the values given to it, You can do either by:

- Iteratively asking inputs from the user(Students already would have done this)
- By using List comprehension

```
def sum_val(list_of_val):
    val = 0
    for ele in list_of_val:
        val += ele

    return val

## Taking inputs from the user
list_of_inp = [int(x) for x in input().split()]

print("The sum is: ",sum_val(list_of_inp) )
```

1 2 3 4 5 6 7 8 9 10
The sum is: 55

- And the same thing could be done using the “*args” method

```
def sum_vals(*args):
    val = 0
    for ele in args:
        val += ele

    return val

print("The value is: ")
print(sum_vals(1,2,3,4,5,6,7,8,9,10))
```

The value is:
55

At this point the student might not be able to see any elegance in this method. But they could be given another example showing a function calculating the simple interest without having parameters explicitly defined for principle, interest and time. This can be shown as

```
def simple_interest(*si):
    simple_int = si[0]*si[1]*si[2]/100

    return simple_int

P = 100
R = 15
T = 3

print("Simple interest is: ", simple_interest(P,R,T))

Simple interest is: 45.0
```

This will cover the functions, loops, list comprehension, default parameters, Variable length parameters/arguments.

Some other concepts such as **immutability** would be discussed for list, tuples and strings.

```
s = 'Hello'

s[1] = 'p'
print(s)
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-13-32b79cdabbe7> in <module>()
      1 s = 'Hello'
      2
----> 3 s[1] = 'p'
      4 print(s)

TypeError: 'str' object does not support item assignment
```

SEARCH STACK OVERFLOW

This example would show string in python are immutable

```
t = (1,4,4,5)

print(t)

t[1] = 6

print(t)
```

```
(1, 4, 4, 5)
-----
TypeError                                Traceback (most recent call last)
<ipython-input-14-713d862e5454> in <module>()
      3 print(t)
      4
----> 5 t[1] = 6
      6
      7 print(t)

TypeError: 'tuple' object does not support item assignment
```

SEARCH STACK OVERFLOW

As tuples are unchangeable, is there any way? A workaround CONVERT it to a list using python in-built **list()**

```
exampleTuple = ("ASHOK", "LARREB", "ASMITA")

new_list = list(exampleTuple)

new_list.append("Jonathan")
print("New List after Appending", new_list)

exampleTuple = tuple(new_list)

print("The tuple after reconvertng from list:", exampleTuple )
```

```
New List after Appending ['ASHOK', 'LARREB', 'ASMITA', 'Jonathan']
The tuple after reconvertng from list: ('ASHOK', 'LARREB', 'ASMITA', 'Jonathan')
```

Q. Dr. Pankaj Jalote has a list of students' information. These are the shortlisted students who are going to be an intern under him for 2 months, to work on a project. The information is in the form of a list. But, this list is a special list. It is a list of dictionaries. This list contains information about students in Physics, Chemistry and, Maths.

Dr. Jalote wants to fetch the marks in each subject as a list. (Maybe he wants to find out how tough or easy the paper was, that years' high school exams).

Help find Dr. Jalote to fetch the marks and complete his objective. For this task we can use our old friend **function** for this purpose.

```
## Let's define the information as a list of dictionary

info = [{"Physics":93, "Chemistry": 98, "Maths": 100},
        {"Physics":88, "Chemistry": 89, "Maths": 99},
        {"Physics":100, "Chemistry": 87, "Maths": 89},
        {"Physics":77, "Chemistry": 81, "Maths": 100}
    ]
```

```
##Let's print the list of dictionary, i.e, the information.
print("the infomation is: ")
for l in info:
    print(l)
```

Output:

```
the infomation is:
{'Physics': 93, 'Chemistry': 98, 'Maths': 100}
{'Physics': 88, 'Chemistry': 89, 'Maths': 99}
{'Physics': 100, 'Chemistry': 87, 'Maths': 89}
{'Physics': 77, 'Chemistry': 81, 'Maths': 100}
```

NOW,

```
## Let's define a function to fetch marks for a given subject

def fetch_marks(list_of_dict, subject):
    ##subject-->the subject name for which marks will be fetched
    ## from all the dictionary present in the list

    toReturn = [X[subject] for X in list_of_dict if subject in X]
    ##List comprehension !!

    return toReturn
```

```
##returning a list of fetched value for a given subject
```

Let's print the marks fetched for the subject, say "Physics"

```
fetchedList = fetch_marks(info, "Physics")
##Called the function with list_of_dict as info,
## and subject as Physics

print(f"The marks fetched for Physics are (in the list form):
{fetchedList}")
```

Output:

```
The marks fetched for Physics are (in the list form): [93, 88, 100, 77]
```

Similarly, marks for "Chemistry" and "Maths" could be fetched

```
fetchedList_chem = fetch_marks(info, "Chemistry")
fetchedList_math = fetch_marks(info, "Maths")

print(f"The marks fetched for Physics are (in the list form):
{fetchedList_chem}")

print(f"The marks fetched for Maths are (in the list form):
{fetchedList_math}")
```

Output:

```
The marks fetched for Physics are (in the list form): [98, 89, 87, 81]
The marks fetched for Maths are (in the list form): [100, 99, 89, 100]
```